EGOI 2025 Editorial - A String Problem

Task Author: Yoav Linhart

July 18, 2025

Introduction

In this problem, we are given $2N$ points, evenly distributed on a circle. There are N straight lines that each connect a unique pair of points. In each step, one endpoint of one line can be changed to another endpoint. The goal is to output the shortest sequence of steps so that, afterward, all lines are parallel to each other, and each endpoint is connected to exactly one line.

Line Notation

From now on, we will also refer to the line that connects the points i and j as the line (i, j).

Subtask 1: Line i is attached to the points i and $i + 1$

In this subtask, every line connects points that are next to each other on the circle. If N is odd, then initially no two lines are parallel, and we have to change all lines except for one (which we can choose arbitrarily). Therefore, the minimum number of steps is $N - 1$. If N is even, then for each line, there is exactly one other line that is parallel to it. Therefore, we have to change all lines except for two, which takes at least $N - 2$ steps.

We can construct a sequence of steps that uses this number of moves (and thus is optimal). We can fix an arbitrary line that we do not change, and make all other lines parallel to it. We choose this line to be the line $(0, 1)$. Therefore, for all $2 \leq i \leq 2N-1$, the point i should be connected to the point $2N-i+1$. Now we can look at pairs of lines that we need to fix. There are $\lfloor \frac{N-1}{2} \rfloor$ such pairs, and the ith pair ($1 \leq i \leq \lfloor \frac{N-1}{2} \rfloor$) contains the lines $(2i, 2i + 1)$ and $(2N - 2i, 2N - 2i + 1)$. We should change this pair of lines to the pair $(2i, 2N - 2i + 1)$ and $(2i + 1, 2N - 2i)$. It is easy to do this using two steps. So in total, we use $2 \cdot \lfloor \frac{N-1}{2} \rfloor$ steps, which is optimal.

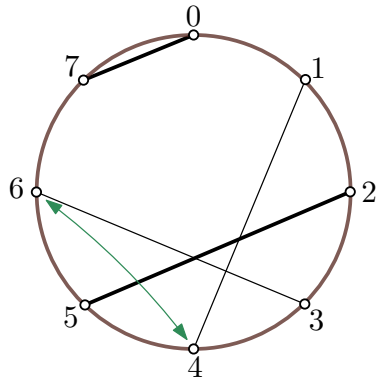
Distinguishing between rotations

We can check if two lines are parallel by considering their endpoints. In particular, a line (a, b) and a line (c, d) are parallel if and only if $a + b \equiv c + d \bmod (2N)$. In other words, the expression $a + b \bmod (2N)$ specifies the *rotation* of the line. We have $2N$ possible line rotations overall, and we want all lines to have the same rotation.

Subtask 2: At most two steps are needed

First, we observe that it is impossible that a single line is not parallel to the rest. Thus, either all lines are parallel to each other (in which case the number of steps is 0) or all lines are parallel to each other except for two (in which case the answer is 2). We can distinguish the cases by computing the rotation of every

line, and by counting the number of times each rotation appears. In the case that two lines are not parallel to the rest, these two lines can be fixed by swapping an endpoint of one line with an endpoint of the other line. This can be seen in the figure below.



Since there are only four possibilities for this, we can simply try all of them and choose the one that makes all lines have the same rotation.

Note that similarly to subtask 1, the minimum number of steps is the number of lines (N), minus the number of times the most common rotation appears. We want to make this work for the general case.

Constructing a solution given a fixed rotation

Given a fixed rotation, we say that a line is *good* if its rotation is the fixed one, otherwise we call it *bad*. To make all lines parallel to the fixed rotation, just changing an arbitrary endpoint of a bad line to make it good does not suffice; we have to be careful that no two lines share an endpoint after all changes are done.

We can avoid this by doing as follows: We pick a bad line ℓ_1, and change one of its endpoints to make it good. This new endpoint is occupied by another line ℓ_2 that is currently bad. Again, we change this endpoint (occupied by the two lines ℓ_1 and ℓ_2) such that line ℓ_2 becomes good and does not share endpoints with ℓ_1. We repeat this process until the new endpoint is not occupied by any other line. After that, we pick a new bad line and start the process again. We do this until all lines are good. This can be done either iteratively or recursively (similar to a DFS), and takes $O(N)$ time. The number of steps we used is N minus the number of lines that initially had the fixed rotation.

Why does the construction always work?

While playing with examples might convince us that the previous construction always works (and this is enough to solve the problem), it is not trivial to understand why. In particular, the unclear part is why we will never reach a new endpoint that is occupied by another *good* line (which would cause us to get stuck).

One way to prove this is using graph theory. Suppose that we already chose the fixed rotation of the parallel lines. Let's model the problem with a bipartite graph – one side contains N nodes that represent the lines in the initial configuration, and the other side contains N nodes that represent the good lines in the final configuration. For each line ℓ in the initial configuration, we add two edges from the node that represents it to the two nodes that represent the good lines to which ℓ can become after changing one of its endpoints. Note that the two edges may connect the same pair of nodes, and this happens if and only if ℓ is initially good. So, we have a bipartite graph with N nodes in each side, and the degree of each node is 2. Our goal is to find a perfect matching in the graph.

We know that any graph that contains only nodes of degree 2 is a collection of disjoint cycles. Since the graph is bipartite, we can infer that all cycles are of an even length!

Now it becomes clear why the construction always works – each disjoint cycle consists of an even number of edges, so if we take every other edge of a cycle, each node in it appears exactly once. Thus, doing so for each cycle results in a perfect matching. Since each edge denotes an initial line and the final line it should become (using one step), considering only the edges in the perfect matching we found results in a correct optimal solution. Note that we have one special case – we do not change anything for cycles of length 2, as they correspond to a line that is initially good.

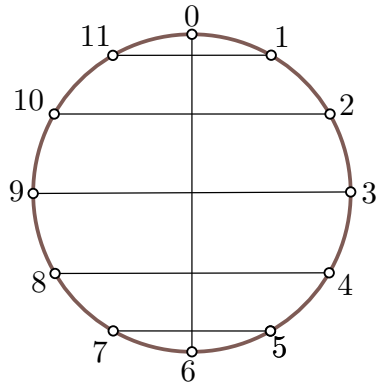
Subtask 3: It is guaranteed that there is a solution where a line connects the points 0 and 1

Since we know that every line should be parallel to the line that connects 0 and 1, we know our fixed rotation and we can directly apply the previous construction to find an optimal sequence of steps in $O(N)$ time.

Subtask 4: $N \leq 1000$

In this subtask, the desired final rotation is not known. We can iterate over all possible rotations and apply the previous algorithm, and take the shortest sequence we found. Since there are $2N$ rotations and an iteration takes $O(N)$ time, this algorithm takes $O(N^2)$ time.

However, there is something we haven't considered: not all rotations are possible – for some rotations, it is impossible to make all lines parallel to that rotation no matter the number of steps. In particular, this is the case if and only if the value of the rotation is even (recall that the value of the rotation of a line (a, b) is $a + b \bmod (2N)$). This is because a line with an even rotation means that the number of points between its two endpoints is odd. Such an example can be seen in the figure below, where it is impossible to make all lines horizontal (or vertical).



So we only check the N odd rotations, which gives us a correct algorithm with a running time of $O(N^2)$.

Full solution

For the full solution, we have to find the optimal rotation without simulating the construction every time. We have already observed that we change each line at most once to become good, and we only change the lines that are initially bad. So, for each fixed rotation, the number of steps is simply the number of bad lines in the initial configuration.

To minimize this, we need to find the largest group of lines that initially have the same rotation. This can be done by counting the number of lines of each rotation and choosing the most common one (the rotation values are from 0 to $2N - 1$ so we can simply use an array for this). Again, we have to be careful to consider

just the odd rotations. If there is no line with an odd rotation, then all lines need to be changed (to an arbitrary rotation). Together with the construction part from subtask 3, we get a solution in $O(N)$ time.

EGOI 2025 Editorial - Currents

Task Author: Brian Lee Jun Siang

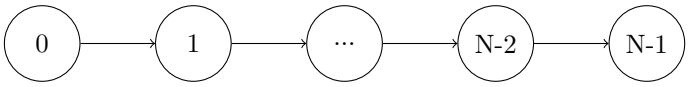
July 18, 2025

Introduction

In the task, a directed graph is given. Exactly once but at any point in time all edges are flipped. You shall calculate the shortest way from each node to the exit (node $N-1$ before the flip or node 0 after the flip), if the flip would be at the least advantageous time.

Subtask 1: $M = N - 1$ and $b_i = a_i + 1$

This subtask restricts the graph to be a path, like described by the graph below.



Looking at a node i. Before the graph is flipped, there is only one possible way to move through the graph, being to move closer to node $N-1$ node by node. After the graph is flipped, there again is only one possible way, being to step by step move closer to node 0.

The observation for this subtask is that the worst moment to get all edges flipped is one step before reaching node $N-1$. Therefore, a formula can be described for the shortest path to a exit when the flipping is in the worst moment: $dist[i] = N - i - 2 + N - 2$. The first part $N - i - 2$ describes the distance to the node $N-2$. The second part $N-2$ describes the way from node $N-2$ to the exit at node 0. This formula can be calculated and printed for each node using a simple loop.

Note that there is an edge case! If the graph only consists of two nodes, the shortest way is always 1 as in this case the worst case would be to go to node $N-1$ (node 1 in this case) and exit there.

Subtask 2: Each cave has a direct channel to cave $N - 1$

Each node having a direct connection to node $N-1$ means that from each node the exit could be reached in one step. Therefore, the worst case for the shortest path is always when the edges get flipped before you even start moving.

For each node, the solution is the shortest way from this node to node 0 in the flipped graph. To calculate this, you can use the original graph and run a Dijkstra algorithm from node 0 resulting in the needed shortest paths.

Note that there again is an edge case! For node 0, the worst case would be to not flip the edges at all and having to reach node $N-1$ in 1 step. Flipping the edges immediately, like for all other nodes, would cause the distance to be 0.

Subtask 3: $N, M \leq 2000$

This subtask is aimed at any sort of solutions with a quadratic run time. Most importantly, this means an inefficient implementation of the full solution.

Subtask 4: Directed Acyclic Graph

The graph in this subtask does not contain any cycles. This allows us to use dynamic programming (DP). Notice that when you are at a node v that is not the exit, there are two options:

1. either the direction reversal happens now. In this case, we take the shortest path to cave 0.
2. The direction reversal does not happen now. Assuming we already know for each successor how long it takes in the worst case to get from there to an exit, we can now move to the successor which has the smallest such time.

The answer for node v is thus the maximum of the distance to 0 and the minimum of the results for the successors plus one. This leads to the following DP formula:

$$dp[v] = \max\left(\min_{w \in successors(v)}(dp[w]) + 1, dist[v]\right)$$

.

$dist[i]$ is the shortest path from node 0. This can be calculated beforehand using Dijkstra's algorithm.

The DP values can be computed in the inverse topological order. This ensures that the results for all successors are computed before the result of a node.

Full Solution

For the full solution, we can use the same insight as in the subtask before, but we no longer have a topological order. In order to still compute the results efficiently, we can adapt Dijkstra's algorithm to compute the results in order of increasing cost: we do not necessarily need to know the results for all successors of a node v for computing the result of v, it suffices if we know the result for the best possible successor!

EGOI 2025 Editorial - IMO

Task Author: Eliška Macáková

July 18, 2025

This problem was inspired by a situation that happened in a certain local math competition. All the results, except one score of one contestant, who used an unusual solution for one of the problems, were published. When she requested the organizers to finally grade it, they argued it would not change her final rank anyway, as there was a big gap between her and the previous person in the ranklist.

The described solutions for the subtasks 1, 2, 3 are independent from the rest of the explanation. However, if you wish to understand the full solution, you should start reading from the solution of subtask 4, as it motivates the solutions for the remaining subtasks and defines some notions that will be reused.

Introduction

In this task, you are given the grading results of a competition per task and contestant, and want to reveal the minimum number of scores such that the correct ranking can be determined.

Test group 1: $N = M = 2, K = 1$

You can solve the subtask with a case analysis or a bruteforce.

Test group 2: $N = 2$

Fix how many scores are revealed for the first and second contestant. Notice that, for the contestant who is supposed to have better score, it is best to grade the problems where she has most points, while for the other contestant, it is best to grade the problems where she has the least amount of points. Check whether this results in a non ambigous comparison between the two contestants, and by checking all the possibilities for how many scores are revealed for the two contestants, find the optimal answer. This can be implemented in $\mathcal{O}(NM^2)$, but asymptotically slower implementations can pass as well.

Test group 3: $N \cdot M \leq 16$

Try all possibilities on which scores to hide and which scores to reveal, then compute the range of points each contestant can have, and check if their ordering is unique (so the ranges may overlap only at the endpoints, and even that only if the comparison according to the criteria in case of equal score evaluates correctly). Time complexity $\mathcal{O}(2^{N \cdot M} \cdot N^2 \cdot M)$.

Test group 4: $K = 1$

The scores of every contestant are either 0 or 1. Let $c_{i,0}$ be the number of 0s the ith contestant has and $c_{i,1}$ be the number of 1s the ith contestant has (observe that $c_{i,0} + c_{i,1} = M$ for every i). When you

decide to reveal y_i scores for the contestant i, the lower bound on her score, l_i, can be any of the numbers $\max(0, y_i - c_{i,0}) \ldots, \min(y_i, c_{i,1})$, and the upper bound on her score, r_i, is going to be equal to $l_i + (M - y_i)$.

This leads to a dynamic programming solution. Sort the contestants according to their final order, the states of the dynamic programming will be (i, S, x) where:

- i = first how many contestants (according to results) have we determined the revealed scores for
- S = lower bound on the score of ith contestant according to our revealed scores
- x = how many scores have we **hid** so far. The number of revealed scores is then simply $i \cdot M - x$.

and $dp[i][S][x] = 1$ if there's a way to do this and 0 otherwise. Transitions from $dp[i][S][x]$ are done by checking all possible l_{i+1}, r_{i+1} (according to the reasoning above), now if $r_{i+1} < S$ or $r_{i+1} = S$ and contestant that should be $i+1$st in the results had higher index original than the one who should be ith, you can update $dp[i+1][l_i][x + (r_i - l_i)]$ ($r_{i+1} - l_{i+1}$ is the number of scores that we should hide for $i+1$ contestant if we want to have the lower bound l_{i+1} and upper bound r_{i+1} on her score, according to the revealed information).

To speed this up, notice that for a fixed i, x, the higher S we have, the better, so we can instead calculate $dp[i][x] = S$, meaning, if we have chosen the scores for first i contestants, and we chose to hide x of them, what is the maximum possible lower bound on score of ith contestant we might have. Now for the transitions, we can first fix the number of scores to hide for $i+1$st contestant, h_{i+1}, and then calculate highest r_{i+1} we can have so that $i+1$ is certainly below i, and then try to update $dp[i+1][x + h_{i+1}]$ with $l_{i+1} = r_{i+1} - h_{i+1}$.

How many scores can we hide? Since we're hiding $r_i - l_i$ scores for contestant i and we know that $M \geq l_1 \geq r_1 \geq l_2 \geq r_2 \geq l_3 \geq \ldots \geq l_n \geq 0$ should hold, then we can get that the total number of hidden scores $= \sum_i r_i - l_i \leq M$. So the number of scores we can hide is asymptotically lower than the total number of scores, which is the motivation behind doing dynamic programming not on the number of revealed scores, but on the number of hidden scores instead.

So we have $N \cdot M$ states and M transitions for each, so the total time complexity is $O(N \log N + NM^2)$ (first factor comes from sorting the contestants by their final score first and original index second in the beginning).

Test group 5: $N \leq 10,000$ and $M, K \leq 10$

Similarly to previous subtask, the main idea is to precalculate for every contestant i the possible lower and upper bounds on their score and then do $dp[i][X]$.

Let's analyze the situation for one contestant, i, and find out what can be the lower and upper bounds on her score. First of all, observe that $r_i = l_i + h_i \cdot K$, where l_i, r_i are the lower and upper bound on the score of contestant i and h_i is the number of hidden scores she has (l_i represents the situation where we grade all the remaining h_i problems with 0 and r_i represents the situation where we grade all the remaining h_i problems with the full score, K).

We can introduce the following dynamic programming to help us find the possible l_i and r_i - let

- j be the prefix of her scores considered (for each score, we are trying to decide whether to hide or reveal it).
- S be the sum of the revealed scores so far
- x be the number of hidden scores so far

Then $t_i[j][S][x]$ is 1 if you can achieve these parameters and 0 otherwise. The transitions from a reachable state $t_i[j][S][x]$ can be done in the following manner - if she has s_{j+1} points on her $j+1$st problem, then if we hide this score, we go to the state $t_i[j+1][S][x+1]$ and otherwise, we go to the state $t_i[j+1][S+s_{j+1}][x]$.

In the end, the bounds l_i, r_i are reachable if and only if $t_i[M][l_i][\frac{r_i - l_i}{K}] = 1$. So we can iterate through all reachable $t_i[M][S][x]$ to find all possible l_i, r_i.

This precalculation can be done in the time complexity $O(M^3K)$ for each i, because we have $O(M \cdot MK \cdot M)$ states and $O(1)$ transitions from each.

Now to solve the original problem, we may adapt the dynamic programming from the previous subtask. The states will be still the pairs (i, x) of the length of the prefix and the number of hidden scores, the values will be the maximum lower bound S we may have, and to transition from $dp[i][x] = S$, fix h_{i+1} and l_{i+1} such that the corresponding r_{i+1} is not too high (less or less than equal to S) and then try to update $dp[i+1][x+h_{i+1}]$ with l_{i+1}. In the beginning, initialize $dp[0][0]$ with $M \cdot K$ and everything else with $-\infty$.

Once again, notice that since in the optimal solution it holds that $M \cdot K \geq l_1 \geq r_1 \geq l_2 \geq r_2 \geq l_3 \geq \ldots \geq l_n \geq 0$, and the number of hidden scores is equal to $\sum_{j \leq i} \frac{r_j - l_j}{K} \leq \frac{M \cdot K}{K}$, so the second dimension of the dynamic programming, x, can go only up to M.

Then the number of states is $\mathcal{O}(N \cdot M)$ and each of them leads to $\mathcal{O}(MK \cdot M)$ transitions, so the total time complexity of this solution is $\mathcal{O}(N \log N + NM^3K)$.

Test group 6: no further constraints

To speed up the solution for the test group 5, we need to make one more optimization. Let m_i be the score of ith contestant if all her scores were revealed. Notice that in the optimal solution, for all i, it must hold that $m_{i+1} \leq l_i$, otherwise, if contestant i would get all 0s for her remaining problems and contestant $i+1$ would get her true scores, they would be incorrectly ordered.

We can then modify the dynamic programming t_i - let

- j be the prefix of scores of contestant i considered (for each score, we are trying to decide whether to hide or reveal it).
- S **be** m_i **minus the sum of her hidden scores so far**
- x be the number of hidden scores so far

then $t_i[j][S][x]$ is 1 if and only if it is possible to reach this state. And from this state you can do transitions to $t_i[j+1][S - s_{j+1}][x+1]$ if you decide to hide the $j+1$st score with the value s_{j+1} and $t_i[j+1][S][x]$ otherwise. Initialize $t_i[0][m_i][0] = 1$ in the beginning and let everything else be equal to 0. The reachable pairs l_i, r_i can still be deduced from $t_i[M]$ just like before.

Now because of the condition that $m_{i+1} \leq l_i$ in any solution, it only makes sense to consider pairs l_i, r_i with $m_{i+1} \leq l_i$, so we don't have to consider states of t_i with $S < m_{i+1}$.

Therefore the time complexity of calculation of t_i is now $\mathcal{O}(M \cdot (m_i - m_{i+1}) \cdot M)$ with the right implementation, summing up to $\mathcal{O}(M^3K)$ for all i.

Now to finish the problem, you can do the same dynamic programming as in test group 5, but you won't consider l_i lower than m_{i+1} when updating for every i, so the total time complexity of this step will be $\mathcal{O}\left(\sum_i (m_i - m_{i+1}) \cdot M^2\right) = \mathcal{O}(M^3K)$.

The total time complexity of this solution is $\mathcal{O}(N \log N + M^3K)$, enough to get full points under the constraints $N \leq 20,000$, $M, K \leq 100$.

EGOI 2025 Editorial - Laserstrike

Task Author: Luca Versari

July 18, 2025

1 Introduction

In this problem, we have two players, the first of whom knows the structure of a tree. The first player needs to choose an order in which to delete the nodes from the tree (always deleting a leaf), then the second player receives, one by one in the same order, the one edge to which those nodes belong.

The second player needs to figure out which node on each edge is the leaf and can use additional bits of information sent by the first player.

2 $N - 1$ bits

Divide all edges $\{a, b\}$ (where $a < b$) into "minor" and "major" edges.

- A "minor edge" is an edge such that, when removing the edge, a is a leaf (and b is not).
- A "major edge" is an edge such that, when removing the edge, b is a leaf (and a is not).

We can then use a single bit per edge, with the bit being 0 if the edge is a minor edge and 1 if it is a major edge.

3 $\approx 0.5N$ bits

We can save half the bits by using the *order of two edges* to encode the type of the next edge.

More precisely, for every edge in an even non-zero position $2i$, we look at the two previous edges that were received. If edge $2i - 2$ compares greater[1] than edge $2i - 1$, the second player treats edge $2i$ as a major edge and as a minor edge otherwise.

The first player might need to swap two edges for this to work. As every tree has always at least two leaves, we can order the edges such that it is always possible to do so, and then decide whether to swap edges in backwards order.

4 1 bit, star

If the tree is a star, we can solve the problem with a single bit. The first player sends a bit that encodes which of the two nodes of the first edge is the center of the star; then, the second player can easily know that the non-center node in every edge is the leaf in that edge.

[1] according to any order, i.e., lexicographic order of the unordered pair of nodes

5 1 bit, path

On a path, we can encode the edges one at a time, starting from one end of the path. For the first edge, we use one bit to encode whether it is a major or a minor edge as usual. For all other edges, we do not need any information: the second player already knows that the node that belongs to the previous edge is the leaf.

6 $2\log_2 N + 1$ bits, star with lines

Let us handle separately the case of paths of length 1 and of length greater than 1.

We first use $\log_2 N$ bits to indicate how many paths have length greater than 1; we then use $\log_2 N$ to indicate how many of those paths have a minor edge as their last edge (going from the star center).

We can then proceed by removing all but the first edge for all of the paths of length greater than 1, applying the path solution. We can tell when we are switching to a new path by the fact that, at that point, the received edge will not share any nodes with the previous edge.

After this step, we are left with a star, and we can use the remaining bit to remove all the remaining edges following the star solution.

7 7 bits, star with lines

Similarly to the previous solution, we can divide edges connected to leaves into three sets:

- edges that are the last edges of paths of length 1
- minor edges belonging to long paths
- major edges belonging to long paths

First, we use one bit per set to identify whether the set is empty.

Within each set, we can sort edges in increasing order. Then we can proceed by first removing the set of edges that contains the largest edge, communicating 2 bits to identify which of the three it is.

- If this set is one of the long-path sets, we also remove most of the path as before; this causes additional edges to be inserted in-between the sorted edges, but this case can be recognized by the fact that the additional path edges share a node with the previous edge, and the leaf edges do not.
- Otherwise, if this set is the length-1 set, we communicate 1 bit to proceed with the star solution for this subset of edges.

Then, we proceed to do the same with the second set of edges, using 1 bit to identify the set. We can notice that we are switching to a new set by the fact that the first new edge will compare *smaller* to the previous leaf-edge, unlike for edges in the same set.

Finally, we remove the remaining edges connected to the star center. Note that at this point we either know the star center (because there was at least one length-1 path), or we didn't use an additional bit during the previous phases and can use an additional bit now to communicate the star center.

8 $\approx 10\log_2 N$ bits, longest distance 10

In this case, we can proceed by iteratively removing all the current leaves; the fact that the longest distance in the tree is 10 guarantees that we will only remove leaves 5 times.

At every iteration, we use $\log_2 N$ bits to represent the number of leaves connected to minor edges, followed by $\log_2 N$ bits to represent the number of leaves connected to major edges.

9 $5(\log_2 N + 1)$ bits, longest distance 10

We can use the idea of sorting two sets of leaves from the solution to star with lines case.

In particular, at every iteration we transmit the total number of leaves at that iteration with $\log_2 N$ bits, then divide the leaves on that layer into minor and major leaves. We then use one bit to indicate whether the first set is the set of major leaves or not. Note that, as we know the sum of the sizes of the sets, we don't need to handle one of the sets being empty.

10 2 bits

Let us divide the nodes of the tree in levels. Level 0 will be the *center* of the tree (one node if the longest path is odd, two nodes if it is even). Each other node will then have a level equal to its distance from one of the level 0 nodes.

Pick a longest path in the tree. This path will have exactly two nodes on each level; we call one half of the path the *starting* path and the second half the *marker* path.

We will remove leaves level by level, starting from the deepest level. On each level, first we will remove all the leaves connected to the *starting* path. This will let us understand when we are switching levels (since on each level there is at least a leaf not connected to the starting path).

We will also remove all the leaves connected to the *marker* path together, either immediately after the starting path or at the end of the level.

For all the other leaves on this level, we proceed as in each iteration of the short-distance case, by splitting leaves in major and minor and encoding which set is the first according to the *position of the marker path leaves in the layer*.

We are left with the task of identifying the starting and marker paths, as well as removing leaves from the deepest level. The first edge we send will be used to identify the starting path, while the second edge (ignoring other edges connected to the starting path) will identify the marker path. We use two bits to encode the ordering on these two paths.

Finally, to remove the remaining leaves on the last level, we proceed as in the other levels, using the ordering of the two starting and marker edges to encode whether the first set of edges are minor or major edges.

11 1 bit

The second player applies the following rules:

1. **First edge**: Use the extra communication bit to determine which node to cut.
2. **If the edge intersects the last removed edge**: remove the node that is *not* in common between the two edges (like in the solution for the star case).
3. **If one of the nodes in the edge has been seen before**: remove the most recently seen node.
4. **Otherwise**: compare to the last edge that was handled by case 1 or 4. If this edge is greater than the previous, then it has the same type (i.e. minor or major). Otherwise, it has the opposite type.

The first player will proceed by iteratively finding all the leaves in the current tree and removing them. The order in which the leaves are removed is as follows:

- If on the current layer there are more leaves that connect to the same node as the leaf that was just removed, remove one of those. Rule 2 ensures that the second player will remove the leaf correctly.

- If we are on the first layer, select a maximum set of leaves that do not share parents, and split them into two sets according to their type. Then, use the one available bit to communicate the type of the first edge, and proceed as usual. Rules 1 and 4 for the second player ensure that they will deduce the correct kind for the edge.
- Otherwise, if this is the first leaf of the current layer, pick any leaf that is **not** connected to the last removed leaf in the previous layer. This is always possible, as every layer contains at least two leaves. This ensures that Rule 3 applies and not Rule 2, and the correct node is removed by the second player.
- Otherwise, remove any leaf. Since the leaf was seen before (as it's not the first layer), the edge doesn't share nodes with the previous edge, and the parent of the leaf was not seen before, this ensures that the correct node is removed by Rule 3.